HACKARUN

// HANDOFF PARA A EQUIPE

GUIA DO PROJETO

Arquitetura, convenções, infraestrutura, inscrições e como contribuir.

Versão 0.3.0 · atualizado em 12 de outubro de 2026
https://github.com/Marcos-Codfy/lp-hackarun

REALIZAÇÃO

UNICATÓLICA

PARCERIA

CAPIM DOURADO SHOPPING

> Documento de handoff: tudo que uma pessoa nova precisa pra rodar, entender, alterar e publicar a landing page. Mantido junto do código. O PDF é gerado a partir deste arquivo com `npm run docs:pdf`. Última atualização: 12/10/2026 (versão 0.3.0 + ajustes em [Unreleased], histórico na seção 18.1).

1. Links rápidos

Item	Valor
Repositório (público)	https://github.com/Marcos-Codfy/lp-hackarun
Branch de produção	main
Hospedagem	Cloudflare Pages, plano gratuito (https://lp-hackarun.pages.dev depois de conectar)
Inscrições	Formulário do site, Google Apps Script (Web App) e Google Sheets
Aviso de privacidade	/privacidade (src/pages/privacidade.astro), linkado no consentimento e no rodapé
Equipes inscritas	/equipes (src/pages/equipes.astro), com foto ou logo; link no header e na LP
Fotos das equipes	Pasta do Google Drive gravada pelo Apps Script (apps-script/Images.gs, seção 9.6)
Node.js	24 LTS (arquivo .nvmrc), mínimo 22.18
Scripts de QA	scripts/qa/ (Playwright + Lighthouse, instalados à parte; seção 13)
Apresentação (README)	README.md (pt-BR) e README.en.md (inglês): a vitrine do projeto
Changelog	CHANGELOG.md (Keep a Changelog + SemVer; seção 18)
Especificação de design	docs/design/design-spec.md (v2, vigente)
Decisões de arquitetura	docs/adr/ (0001 a 0007)
Stack e padrões (PDF)	docs/pdf/HackaRun_LP_Stack_e_Padroes.pdf

2. O que é

Landing page do **HackaRun 2026**, o hackathon dos cursos de Engenharia de Software e Inteligência Artificial da **Unicatólica**, com parceria do **Capim Dourado Shopping**. Evento de 11 a 14 de novembro de 2026, com minicursos, mentoria de egressos, corrida de missões (os NPCs dão pistas) e premiação no shopping com telão e banda.

Objetivos da página: explicar o evento, mostrar a programação e as missões, mostrar as equipes confirmadas (na página `/equipes`, com a foto ou a logo de cada time) e **converter em inscrições**. Slogan oficial: **"Desvende. Programe. Conquiste."**

A versão atual é a **v2 cinematográfica** (ADR 0006): identidade oficial, pós-processamento real e um objeto 3D novo para cada seção. A v1 ficou só no histórico do Git.

Situação em 11/10/2026: design v2 **aprovado**, código publicado no GitHub (`main`) e Lighthouse mobile com Performance 95 e Acessibilidade, Boas práticas e SEO em 100. O que falta para

abrir as inscrições está na seção 19.

Em 12/10/2026 entraram os ajustes pedidos pela organização, ainda sem versão fechada (`[Unreleased]` no changelog): o site não tem mais som, o clímax da inscrição ficou mais curto e calmo, e as equipes ganharam a página `/equipes`, com a foto ou a logo que cada uma sobe na inscrição. Resumo na seção 18.1. As medições do Lighthouse acima são de 11/10, antes desses ajustes.

3. Decisões principais

Decisão	Escolha	Motivo curto
Framework	Astro (site estático) + TypeScript	HTML pronto no build, quase zero JS inicial, ótimo LCP e SEO
3D	Partículas procedurais (Three.js + GLSL), 8 objetos	0 KB de vídeo/modelo, nítido em qualquer tela, muda com o scroll
Visual	Pós-processamento (bloom, lente) + câmera coreografada	Pedido do cliente: impacto de cinema, não "correto demais"
Identidade	Logo, paleta e fontes oficiais; til da Orbitron corrigido	Fidelidade ao manual da marca; "NÃO" legível em título grande
Animação	GSAP + ScrollTrigger, Lenis, LineSplit próprio	Padrão de mercado pra scroll storytelling, gratuito
Hospedagem	Cloudflare Pages (grátis)	Banda e requisições ilimitadas pra estático, CDN no Brasil
Inscrições	Google Sheets via Apps Script	Gratuito, a organização já usa Google, planilha vira o painel
Fotos	Google Drive + CDN lh3.googleusercontent.com	Grátis, fica com a organização, só aparece depois da confirmação
Arquitetura	Clean Architecture + DDD + MVVM	Camadas claras, regras testáveis, fácil de dividir entre pessoas
QA	Playwright + Brave instalado, Lighthouse (scripts/qa)	Captura visual em várias larguras e métricas reais de GPU

Detalhes e alternativas descartadas estão em `docs/adr/`.

4. Como rodar

```
git clone https://github.com/Marcos-Codfy/lp-hackarun.git
cd lp-hackarun
npm ci
cp .env.example .env
npm run dev
```

Abra `http://localhost:4321`. Sem `PUBLIC_REGISTRATION_ENDPOINT` o formulário roda em **modo simulado** no ambiente de desenvolvimento: não grava em lugar nenhum, e as equipes inscritas na simulação aparecem já confirmadas, com a foto, em `/equipes` e na LP enquanto a aba estiver aberta (`sessionStorage`, chave `hackarun:dev:simulated-teams`; apague a chave para

ver os estados vazios).

Script	O que faz
npm run dev	Servidor local com hot reload
npm run build	astro check + build de produção em dist/
npm run preview	Serve o dist/ localmente
npm run lint / lint:fix	ESLint (inclui as regras de dependência entre camadas)
npm run format	Prettier em tudo (a CI roda format:check)
npm test / test:watch	Testes unitários (Vitest)
npm run validate	lint + testes + build (o mesmo que a CI roda)
npm run docs:pdf	Gera os PDFs de documentação (e copia pra DOCS_OUT_DIR)

Parâmetros de URL para testar:

- `?quality=low`, `?quality=medium` ou `?quality=high`: força o tier do 3D e trava o ajuste automático por FPS.
- `?now=2026-11-14T16:00:00-03:00`: desloca o relógio do site (troca de marcos do contador, estado da inscrição).

5. Estrutura de pastas

```
src/
  domain/              regras de negócio puras (entities, value-objects, services, repositories)
  application/         casos de uso, DTOs e portas (Clock, CalendarExporter, RegistrationGateway,
                       PreferenceStore, ImageCodec)
  infrastructure/      adaptadores: JSON, repositórios, Apps Script, imagem, storage, calendário
    content/           >>> conteúdo editável do site (event.json, stations.json, faq.json...)
    image/             CanvasImageCodec: abre, redimensiona e comprime a foto da equipe no navegador
  assets/
    brand/             logo oficial do HackaRun e logos brancas dos parceiros (PNG)
    fonts/             Orbitron com o til redesenhado (gerada por script)
  presentation/
    app/               App.boot (LP), LightApp.boot (páginas leves), AppConfig, AppContext e AppEvents
                       (contrato do EventBus)
    layouts/           BaseLayout (LP, com o canvas 3D), PageLayout (páginas leves) e parts/ (head, fontes,
                       SEO e aurora de fundo em comum)
    sections/          uma seção por arquivo; subcomponentes em pastas (hero/, missions/, teams/...)
    components/        ui/, layout/, hud/, countdown/, registration/, teams/, partners/,
                       npc/, motion/ (pausar animações), seo/
    controllers/       header, menu mobile, nav spy, trilho do HUD, theme/ (aurora)
    view-models/       ViewModels (MVVM): estado e lógica de tela, sem DOM
    views/             Views (MVVM): ligam DOM e ViewModel
    formatters/        textos e datas da interface
    motion/            core/ (Lenis, ScrollTrigger, SceneDirector, AnchorNavigator) e animators/
                       (entradas, raios)
    experience/        3D, carregado sob demanda:
      shapes/          uma ShapeStrategy por objeto + utilitários de traço e amostragem
      shaders/         core.glsl, shapes/*.glsl (um por objeto), partículas, estrelas, bokeh, lente
      particles/       ParticleField, ParticleMaterial e ColorGrade (cor por cena)
      post/            pipelines de render, bloom, passe cinematográfico e looks por cena
      camera/          enquadramentos, voos entre cenas e câmera na mão
      layers/          estrelas, bokeh e anel de choque
      scene/           SceneMixer, SceneLayout, ChargeController e ShapeSpin
      quality/         tiers, detector de GPU e ajuste por FPS
      input/           ponteiro (parallax e repulsão)
```

```
    styles/           tokens.css (design tokens), base.css, deferred-sections.css (render adiado)
                      e motion-pause.css (o que para com "Pausar animações")
  shared/             utilitários sem regra de negócio (Observable, EventBus, Ticker, math, dom)
  composition/        composition roots: server.ts (build) e client.ts (browser)
  pages/              rotas do Astro (index, equipes, privacidade, 404, .ics)
apps-script/          código do Google Apps Script (API de inscrições e fotos no Drive) + passo a passo
public/               servido como está: og-image.jpg, favicon, _headers, robots
scripts/
  fonts/              patch_orbitron_tilde.py: til da Orbitron
  docs/               gerador dos PDFs, build_pdf_assets.py e assets/ (fontes TTF e logos)
  qa/                 QA visual e de performance (Playwright + Lighthouse), com package.json próprio
tests/                testes unitários espelhando as camadas
docs/                 guia, design spec, asset brief, ADRs, manifesto da stack e PDFs gerados
design/references/    referências visuais e o vídeo teaser (não vão pro site)
```

Páginas geradas no build:

Rota	Arquivo	O que é
/	pages/index.astro	A landing page
/equipes	pages/equipes.astro	Equipes confirmadas com foto ou logo; página leve, sem 3D
/privacidade	pages/privacidade.astro	Aviso de privacidade (LGPD) com a versão vigente do aviso
/404	pages/404.astro	Página de erro, no mesmo visual
/hackarun-2026.ics	pages/hackarun-2026.ics.ts	Agenda com todas as atividades (botões da área de inscrição)

O "Adicionar à agenda" aparece no aviso de inscrições em breve, no lobby do formulário e na tela de sucesso. O botão que ficava na programação saiu a pedido da organização. Todas as páginas entram no sitemap (a integração só exclui 404 e 500).

6. Arquitetura

6.1 Camadas e regra de dependência

As dependências sempre apontam para dentro: `presentation` -> `application` -> `domain`. `infrastructure` implementa as interfaces (portas) declaradas em `domain`/`application`. Quem junta tudo são os **composition roots** em `src/composition`. O ESLint **bloqueia** imports proibidos (`eslint.config.js`):

Camada	Pode importar	Bloqueado
domain	nada de fora	outras camadas e frameworks (three, gsap, lenis, astro)
application	domain	infrastructure, presentation, composition e frameworks
presentation	application (casos de uso, DTOs e portas)	infrastructure: o adapter concreto é ligado em composition
shared	nada	qualquer camada (é utilitário sem regra de negócio)

Na prática: uma View ou ViewModel que precisa de armazenamento, imagem ou rede recebe a **porta** ou o caso de uso (ex.: `PreferenceStore`, `RegistrationGateway` e `ImageCodec` em `application/ports`, este último dentro do `PrepareTeamImage`) pelo construtor, e o `client.ts` decide qual implementação entra.

6.2 Fluxo no build (SSG)

1. `src/pages/index.astro` chama `serverContainer().getLandingPage.execute()`.
2. O caso de uso lê os repositórios estáticos (JSON em `infrastructure/content`), que passam pelos **mappers** e viram entidades/aggregate. Dado inválido quebra o build (fail fast).
3. A aplicação devolve **DTOs** serializáveis, e as seções renderizam HTML puro.
4. Um JSON (`script#app-config`) leva pro browser só o necessário: marcos do contador, dados da inscrição e o endereço da lista de equipes ao vivo.

6.3 Fluxo no browser

`App.boot()` espera o primeiro frame pintado (o hero, que é o LCP, aparece antes de qualquer JS pesado) e monta tudo por uma fila (`app/BootQueue.ts`): um passo curto por tarefa, cedendo a thread entre eles (`scheduler.yield`, ou `setTimeout` 0 onde não existe). Cada passo aparece na trilha Timings do DevTools como `boot:<nome>`, e um passo que quebra não derruba os seguintes. Ordem:

1. `mountMotion`: Lenis (smooth scroll), ScrollTrigger, o **SceneDirector** (acompanha as seções com `data-scene`) e o `AnchorNavigator` (links `#secao`). Se GSAP ou Lenis quebrar, a página segue no scroll nativo (`NativeScroll`).
2. `mountViews`: Views MVVM (pausar animações, contador, HUD, tokens NPC, CTAs de inscrição, equipes ao vivo e formulário). Funcionam sem 3D.
3. `mountControllers`: controladores de DOM (header e menu primeiro, nav spy, trilho do HUD, aurora). Não usam GSAP, então um animador quebrado não leva eles junto.
4. `mountAnimators`: o pin das missões e as entradas animadas, que só preparam o setup pesado quando a seção chega perto da tela (`motion/core/ApproachWatcher.ts`); os títulos se dividem em linhas pelo `LineSplit` próprio (`motion/animators/heading/LineSplit.ts`).
5. Segunda fila, com o layout já no lugar: trilha de cenas, `SceneHoldWatcher` (carga do globo final) e, depois do `load` com o navegador ocioso, o chunk do 3D (`mountExperience`). Sem WebGL2, com `save-data` ou se algo falhar, ele emite `experience:fallback` e o brilho em CSS continua.

Páginas leves (`/equipes`): usam o `PageLayout` e o `LightApp.boot()`, que roda a mesma `BootQueue` só com header e menu, "Pausar animações" e a lista de equipes ao vivo (`TeamsView`). Nada de GSAP, Lenis, ScrollTrigger nem three.js: o menu trava o scroll nativo (`NativeScroll`) e, no lugar do objeto 3D, fica um halo de HUD estático em CSS. Os links da LP no header e no rodapé viram `/#secao` fora da home (`navLinksFor` em `components/layout/navLinks.ts`), e o item da página atual recebe `aria-current="page"`. `/privacidade` e `/404` ainda usam o boot completo e podem migrar para o `PageLayout` depois.

6.4 MVVM

- **ViewModel** (`presentation/view-models`): estado em `Observable`, chama casos de uso, não toca no DOM. Testável.
- **View** (`presentation/views`): assina o ViewModel, atualiza o DOM e repassa eventos do usuário.
- **Model**: casos de uso + domínio.

6.5 Controladores de DOM

Em `presentation/controllers`, montados por `mountControllers` (não são MVVM e funcionam sem GSAP): `HeaderScrollController` (header de vidro que reage ao scroll), `MobileMenuController` + `FocusTrap` + `ScrollLock` (overlay do menu no celular), `NavSpyController` (link ativo), `HudRailController` (tiques do trilho do HUD) e `SceneThemeController` (cores da aurora por cena). Os links internos ficam com o `AnchorNavigator` (`motion/core`): rola com o driver de scroll, atualiza o hash sem pulo e leva o foco para a seção.

6.6 Eventos entre módulos

Os módulos conversam pelo `EventBus` tipado (`presentation/app/AppEvents.ts`):

Evento	Quando
scene:enter	Uma seção com data-scene chegou ao centro da tela
experience:ready	O 3D desenhou o primeiro frame (informa o tier)
experience:fallback	O 3D não vai rodar (sem WebGL, save-data, erro de carga ou de shader)
experience:charging	A seção com data-scene-hold entrou: o globo e a barra do CTA começam a carregar
experience:charged	Fim da carga: pulso suave do globo e o CTA acende (emitido mesmo sem WebGL)
motion:paused	O usuário pausou ou retomou as animações (seção 6.7)
npc:found	Um token NPC foi encontrado
registration:intent	Clique num CTA de inscrição (guarda a origem)
registration:submitted	Inscrição enviada com sucesso (protocolo)

O terminal do NPC e o protocolo da inscrição continuam "digitando" na tela, mas em silêncio: o site não tem som.

6.7 Pausar animações (WCAG 2.2.2)

Tudo que se mexe sozinho por mais de 5 s (3D, raios do logo, loops em CSS) para com o botão **Pausar animações**: no header, entre a navegação e o "Participar" (`components/motion/PauseMotionToggle.astro`, ícone com tooltip), e no menu do celular (`PauseMotionSwitch.astro`, com texto). Os dois têm `data-motion-toggle`, então andam juntos. Na página `/equipes` o botão também existe e pausa os loops em CSS (lá não há 3D).

- **Estado:** `MotionPreferenceViewModel` (`view-models/motion`). Começa rodando e a escolha fica salva no navegador (porta `PreferenceStore`, chave `motion`). Com `prefers-reduced-motion` o sistema já parou tudo: o botão fica travado em "pausado" (`aria-disabled`) e não grava nada.
- **DOM:** `MotionPreferenceView` liga os botões (nome acessível fixo, estado no `aria-pressed`, tooltip com a ação do próximo clique, Esc esconde o tooltip) e põe a classe `is-motion-paused` no `<html>` (`MotionPauseSignal`). O evento `motion:paused` sai a cada troca.
- **Quem obedece:** o CSS lê a classe direto; o `FrameLoop` do 3D congela o relógio (desenha um frame parado e, ao retomar, segue dali sem salto) e os raios do logo param. Os dois leem o `MotionPauseSignal` ao montar, então já nascem no estado certo.
- **Loops em CSS:** `styles/motion-pause.css` tem a **lista explícita** de animações infinitas que recebem `animation-play-state: paused`. Um "pausa tudo" travaria no quadro zero as entradas de uma vez só (hero, menu) e o conteúdo sumiria. **Criou um loop infinito novo em CSS? Inclua a classe nessa lista.** Spinner de envio fica de fora (é status, não enfeite).
- Sem JavaScript os botões continuam `hidden`: não haveria o que pausar além do CSS.

7. Experiência 3D

7.1 Cenas e objetos

Um único `THREE.Points` guarda um alvo de posição por cena; o morph inteiro acontece no vertex shader.

Cena	Seções	Objeto	Arquivos
core	Hero e Inscrição	Globo de energia em fitas, borda acesa por fresnel	CoreShape.ts, core.glsl
orbit	O evento	Cronômetro-giroscópio: mostrador, ponteiro e anéis em cardã	OrbitShape.ts, orbit.glsl
enigma	NPCs	"?" extrudado com faixa tracejada e auréola de HUD	EnigmaShape.ts, enigma.glsl
map	Missões	Mapa em holograma: plataforma, rota em espiral, 8 faróis	MapShape.ts, map.glsl
terrain	Programação e Mentores	Terreno synthwave: grade rolando, montanhas e sol em faixas	TerrainShape.ts, terrain.glsl
stage	Grande final	Palco: telão de LED, treliça, canhões de luz e plateia	StageShape.ts, stage.glsl
trophy	Prêmios	Troféu de torno em linhas, alças e estrela	TrophyShape.ts, trophy.glsl
dust	Equipes e FAQ	Galáxia distante (bojo, dois braços, estrelas soltas)	DustShape.ts, dust.glsl

Os `.ts` ficam em `experience/shapes/` e os `.glsl` em `experience/shaders/shapes/` (o globo em `experience/shaders/core.glsl`). Funções comuns das formas estão em `shaders/shapes/helpers.glsl`.

7.2 Como o morph funciona

1. Cada seção tem `data-scene="<chave>"`. O `SceneDirector` transforma o scroll num índice contínuo (`scenes.progress`, ex.: 3,4 mistura a cena 3 e a 4), mais a velocidade do scroll.
2. O `SceneMixer` converte o índice no trecho atual (forma de origem, de destino e progresso suavizado).
3. Cada `ShapeStrategy` gera a nuvem de pontos estática da forma (uma vez, com semente fixa). A geração roda num **Web Worker** (`shapes/shapes.worker.ts`, chamado pelo `ShapeSource`), que devolve os buffers por transferência enquanto a GPU compila os shaders; sem worker, o mesmo cálculo roda na thread principal em fatias (`ParticleJob`). Todas as formas usam a mesma permutação, então a partícula "i" ocupa o mesmo lugar relativo em toda forma e a troca não vira chuvisco.
4. No vertex shader, cada forma tem `shape<Nome>(p)` (deformação viva: giro, ondas, pulsos) e `<nome>Heat(p)` (calor que puxa a cor pra crista ou pro acento). A troca é uma onda que varre a tela, e no meio do voo as partículas escorrem por um fluxo contínuo.
5. O `ParticleMaterial` monta o shader com `defines`: cena ausente na página nem declara o atributo (`HAS_<CENA>`), e o compilador remove o ramo inteiro.

7.3 Pipeline de pós-processamento

Em `experience/post/`, escolhido pelo nível de efeitos do tier:

- `ComposerPipeline` (tiers médio e alto): `RenderPass` -> `ScaledBloomPass` (UnrealBloomPass com resolução própria e tinta por cena) -> `CinematicPass` (`shaders/cinematic.frag.glsl`: onda de choque discreta, só uma tinta ciano e um pouco de refração; zoom blur da travessia, aberração cromática, exposição com ombro suave, vinheta e grão). Não há mais flash branco. Buffers HalfFloat guardam o excesso de luz.
- `DirectPipeline` (tier baixo): render direto; o sprite da partícula carrega um halo "fake" e o anel de choque vem da malha 3D.
- `PostDirector` mistura os `SCENE_LOOKS` (bloom, raio, limiar, tinta, exposição, estrelas, bokeh) com os mesmos pesos do morph e soma os eventos (carga, pulso, brilho residual, travessia). O que a carga soma tem teto: `MAX_CHARGE_BLOOM` (0,3 no bloom) e `MAX_CHARGE_EXPOSURE` (8% na exposição). O `StageRenderer` troca de pipeline em tempo de execução.

7.4 Câmera, camadas e cor

- **Câmera** (`camera/`): `CameraShots` define o enquadramento de cada cena (truck, crane, dolly, mira, FOV, órbita), `FlightPath` a coreografia de cada travessia (ex.: atravessar o globo rumo à próxima cena) e `Handheld` a deriva lenta de "câmera na mão" (soma de senos, sem tremor). `PointerTracker` dá parallax e repulsão no desktop.
- **Camadas** (`layers/`): `StarField` (céu ao fundo), `BokehLayer` (luzes desfocadas na frente) e `ShockwaveRing` (anel do clímax), orquestradas pela `Atmosphere`.
- **Cor**: `particles/ColorGrade.ts` dá a cada forma fundo, corpo, crista e acento da paleta oficial. Atrás do canvas, a **aurora** (`BaseLayout`) muda de cor por cena via `SceneThemeController` (`controllers/theme/sceneThemes.ts`); as cores são `@property <color>`, então a troca interpola. O canvas é opaco e soma sobre a aurora em `mix-blend-mode:`

`screen`.

7.5 Clímax da inscrição

A seção `#inscricao` tem `data-scene="core"` e `data-scene-hold="900"`. Quando ela entra, o `SceneHoldWatcher` emite `experience:charging` e conta 0,9 s **em tempo** (o scroll nunca é travado). Atributo vazio ou inválido cai no `DEFAULT_HOLD_MS` (900 ms, via `holdDuration()`). Com `prefers-reduced-motion` a carga é instantânea.

- **Carga:** o `ChargeController` junta energia em curva (fitas um pouco mais agitadas e quentes) e a câmera se aproxima devagar. Ao mesmo tempo, o `ChargeRevealView` põe `is-charging` e `--charge-ms` na seção, então a barra do CTA enche no mesmo tempo do globo.
- `experience:charged`**:** o globo solta um **pulso suave**. A luz (`swell`) sobe em 0,22 s e apaga devagar, o pulso radial é quase igual para todas as partículas (o globo respira inteiro, as fitas não viram nuvem), o anel abre até cerca de 2,6 vezes o raio do globo e some, e tudo assenta num brilho calmo. Sem flash branco e sem tremor de câmera. O CTA acende no mesmo evento (`charge-glow`).
- **Segurança:** se o evento não chegar, o CTA aparece sozinho em carga + 2,5 s (ou 6 s depois de a seção entrar na tela). Sem WebGL o evento sai do mesmo jeito. Se a pessoa sai da seção no meio, a carga descarrega.
- **Linha de status** (`.charge__status`): fica numa placa escura com contorno fino para manter o contraste sobre o anel, e as duas partes, separadas por " · ", empilham abaixo de 640 px. Mudou o `RegistrationCopy.statusLine`? Mantenha o " · " como separador.

7.6 Tiers de qualidade

Tier	Partículas	DPR máx.	Pós	Estrelas	Bokeh	Quem recebe
low	9.000	1	sem bloom (halo fake)	900	12	GPU por software, celular sem GPU forte
medium	22.000	1,5	bloom em meia resolução	1.800	22	celular capaz, notebook com GPU integrada
high	42.000	2	bloom inteiro	3.200	30	GPU dedicada ou Apple M; desktop com 8+ núcleos

- O tier inicial vem do `QualityDetector` (nome da GPU, núcleos, memória, tela e tipo de ponteiro).
- O `AdaptiveQuality` mede o FPS: abaixo de 45 por ~2 s corta um degrau do pós (inteiro -> meia -> nenhum), depois o DPR, depois o tier. Nunca sobe de volta.
- O render pausa com a aba oculta e congela com "Pausar animações" (seção 6.7). Com `prefers-reduced-motion`, a cena troca inteira e o loop desenha sob demanda.

7.7 Como adicionar uma cena

1. Adicione a chave em `SceneKey` (`presentation/app/AppEvents.ts`). O TypeScript passa a apontar todos os `Record<SceneKey, ...>` que faltam: `ColorGrade.ts`, `SceneLooks.ts`, `CameraShots.ts`, `SceneLayout.ts`, `ParticleField.ts` (nome do atributo) e `sceneThemes.ts` (aurora).
2. Crie a `ShapeStrategy` em `experience/shapes/<Nome>Shape.ts` (`generate(count, random)` devolve xyz em sequência, dentro de `SHAPE_RADIUS`) e registre em `ShapeFactory.createDefault()`. Prefira os utilitários `strokes.ts`, `PointBrush` e `PolylinePath` (traços finos e contínuos em qualquer tier).
3. Crie `experience/shaders/shapes/<chave>.glsl` com `vec3 shape<Nome>(vec3 p)` e `float <chave>Heat(vec3 p)`. Constantes compartilhadas com o `.ts` ficam espelhadas nos dois arquivos, com comentário avisando.
4. Declare o atributo em `shaders/common.glsl` (`#ifdef HAS_<CHAVE>` / `attribute vec3 a<Nome>;`), adicione o bloco `ADD_SHAPE` em `particles.vert.glsl` e inclua o `.glsl` e a chave (`ALL_KEYS`) no `ParticleMaterial.ts`.
5. (Opcional) Uma curva de travessia em `camera/FlightPath.ts`.
6. Na seção: `data-scene="<chave>"` e `data-checkpoint="<rótulo do HUD>"`.
7. Confira com `?quality=low` e `?quality=high`, no celular e no desktop (seção 13).

8. Editar conteúdo sem programar

Tudo em `src/infrastructure/content/`. Edite, rode `npm test` (os testes validam o conteúdo real) e abra um PR.

Arquivo	O que controla
event.json	Datas, locais, marcos do contador e inscrição (endpoint, closesAt, capacity, teamSize)
stations.json	As 8 missões e a pontuação (a soma dos critérios precisa dar 100)
speakers.json	Instrutores, mentores, palestra e banca (name: null = card "a revelar")
prizes.json	Prêmios (reward: null = "a revelar")
partners.json	Realização e parceiros (nome, papel e link)
faq.json	Perguntas frequentes
teams.json	Lista reserva de equipes (as oficiais vêm da planilha, ao vivo)

Textos fixos de seção (títulos, chamadas) ficam nos próprios `.astro` em `presentation/sections/`.

9. Inscrições (Google Sheets)

Fluxo: formulário (com a foto ou logo, no modo equipe) -> validação no navegador (regras do domínio) -> `POST` para o Web App do Apps Script -> nova linha na aba **Inscricoes** com protocolo `HR26-XXXXXX` e a foto numa pasta do Drive (seção 9.6). Para a equipe aparecer no site, a organização muda a coluna **Status** para `confirmado`: ela entra em `/equipes` e no contador da LP. O site lê `?action=teams`, que expõe **só** nome da equipe, curso, quantidade de pessoas e o id da foto.

Passo a passo completo em `apps-script/README.md`. Depois de publicar o Web App, coloque a URL em `PUBLIC_REGISTRATION_ENDPOINT` (variável de ambiente na Cloudflare) ou em `registration.endpoint` no `event.json`.

O Apps Script tem quatro arquivos: `Code.gs` (rotas `doPost`/`doGet`, planilha e protocolo), `Validation.gs` (regras do servidor), `Limits.gs` (prazo, capacidade e limite de envios) e `Images.gs` (foto da equipe no Drive). Os testes rodam os quatro de verdade no Node, com dublês dos serviços do Google (planilha, cache, lock, propriedades e Drive): `npx vitest run tests/infrastructure/apps-script`.

> **Mudou algum `.gs`? Publique uma nova versão do Web App junto com o deploy do site** (Implantar > Gerenciar implantações > Editar > Nova versão). O script atual só aceita o aviso vigente (`CONSENT_VERSION`) e exige a foto nas inscrições de equipe, então site e script precisam subir juntos.

9.1 Prazo e capacidade em tempo de execução

O build fecha o formulário pelo `event.json` (`closesAt`, `capacity`), mas qualquer um pode chamar a URL do Web App direto. Por isso o `Limits.gs` confere **a cada envio** as Propriedades do script e responde `closed` quando não dá mais:

Propriedade	O que faz
CLOSES_AT	Prazo final, data ISO com hora e fuso (2026-11-08T23:59:00-03:00)
CAPACITY	Máximo de equipes: conta as linhas Equipe que não estão recusado
INSCRICOES_FECHADAS	sim fecha na hora, sem novo deploy do site

- Lotou, fecha pra todo mundo (equipe e individual), igual ao site. Recusar uma equipe na coluna Status libera a vaga.
- Valor inválido (`CAPACITY=trinta`) **fecha** as inscrições e grava o erro no log: melhor errar pro lado seguro.
- `URL/exec?action=health` responde `open: true/false` sem ler a planilha: serve pra conferir a configuração.
- Mantenha `CLOSES_AT` e `CAPACITY` iguais a `registration.closesAt` e `registration.capacity` do `event.json`.

9.2 Limites de envio

- **Por e-mail:** até 10 tentativas a cada 10 min por e-mail do líder (guardado como hash no `CacheService`, nunca em texto). Cobre os reenvios automáticos do site e segura quem fica testando se um e-mail já está inscrito.
- **Global:** 120 envios por minuto. O e-mail esbarra no próprio limite antes de contar no global, então um script repetindo o mesmo e-mail não tranca a fila de todo mundo.
- Resposta `rate-limited`: o `RetryingRegistrationGateway` tenta de novo com espera crescente (até 3 tentativas, só para `network`, `server` e `rate-limited`).

9.3 Consentimento (LGPD)

- O checkbox do formulário linka o aviso em `/privacidade` (abre em nova aba), que também está no rodapé.
- O envio leva `consentVersion` (a constante `CONSENT_VERSION` de `application/use-cases/SubmitRegistration.ts`, hoje `2026-10-v2`) e `consentAt`. O servidor exige os dois, **só aceita a versão vigente** (a mesma constante, espelhada no `Validation.gs`; um teste confere) e grava na coluna **Consentimento**: `versão · data/hora ISO do aceite`. Sem isso a inscrição não entra.
- No modo equipe, o texto do aceite inclui a autorização para publicar a foto ou logo e a declaração de que todos que aparecem nela concordaram. O aviso tem uma seção só para a foto (finalidade, base legal, link público do Drive, retenção e como pedir a retirada).
- **Mudou o texto do aviso? Suba o `CONSENT_VERSION` nos dois arquivos** e publique o script junto com o site. A página `/privacidade` mostra a versão vigente.

9.4 Idempotência (reenvio sem linha duplicada)

- Cada sessão do formulário gera uma **chave de envio** (UUID, `view-models/registration/idempotencyKey.ts`) que vale para todo reenvio até a inscrição entrar (`SubmissionSession`).
- No servidor, mesma chave + mesmo e-mail devolve o **mesmo protocolo**: primeiro pelo cache (10 min) e, depois disso, pela coluna **Chave de envio** gravada na linha. Mesmo e-mail com outra chave é `duplicate`.
- Planilha criada antes da coluna nova: o script completa o título dela na primeira gravação.

9.5 Outras proteções

Honeypot e tempo mínimo de preenchimento (3 s: `MIN_FILL_MS` no `SubmitRegistration.ts` e `MIN_ELAPSED_MS` no `Validation.gs`, um teste confere), validação completa no servidor (nome, e-mail, WhatsApp, curso, tamanho da equipe, e-mails repetidos), trava de concorrência (`LockService`), proteção contra fórmula injetada na planilha (`=`, `+`, `-`, `@`) e cache de 60 s na lista de equipes (o `onEdit` limpa o cache quando a coluna Status ou a Imagem muda).

9.6 Foto ou logo da equipe

Decisão registrada na ADR 0007. Na inscrição de equipe o campo **Foto ou logo da equipe** é obrigatório (`components/registration/TeamImageField.astro` e `TeamImagePreview.astro`).

No navegador:

- Aceita JPEG, PNG ou WebP de até 10 MB, com o lado menor de pelo menos 200 px. As regras ficam no value object `TeamImage` (`TEAM_IMAGE_RULES`, no domínio).
- O caso de uso `PrepareTeamImage` usa a porta `ImageCodec` (o adapter é o `CanvasImageCodec`, com `createImageBitmap`, orientação EXIF e `OffscreenCanvas` quando existe): reduz para 800 px no lado maior e tenta WebP, depois JPEG, baixando a qualidade e depois o tamanho até caber em ~300 KB. Passou de 400 KB, recusa.
- Logo com fundo transparente ganha um fundo sólido que contrasta com o traço, para não sumir no card escuro.
- Prévia com "Trocar imagem", arrastar e soltar, erros no resumo de erros do formulário (`aria-invalid`, `aria-describedby`) e status em região `aria-live`. O `TeamImageSlot` (view-model) descarta resultado atrasado de uma imagem trocada no meio.
- A imagem vai em base64 dentro do mesmo `POST text/plain`. No reenvio vão os mesmos bytes com a mesma chave de envio (nada é reprocessado), e clicar em enviar com a imagem ainda em preparo espera ela ficar pronta. O envio espera até 30 s (`AppsScriptRegistrationGateway`).

No Apps Script (`Images.gs`):

- Confere o base64, o teto de 400 KB (`MAX_IMAGE_BYTES`, igual ao `TEAM_IMAGE_RULES.maxBytes`; um teste confere) e o formato pelos primeiros bytes. O tipo declarado precisa bater com eles.
- Grava só depois das checagens de reenvio e de e-mail repetido, então um retry nunca cria um segundo arquivo. O nome é sempre o protocolo (`HR26-ABC234.webp`).
- Pasta: a Propriedade `FOTOS_PASTA_ID`. Sem ela, o script cria `HackaRun 2026 · Fotos das equipes` uma vez e guarda o id. A função `prepararFotos()`, rodada uma vez no editor, autoriza o Drive e cria a pasta.
- Cada arquivo fica como "Qualquer pessoa com o link · Leitor". Se o domínio bloquear link público, ou se o Drive falhar, a inscrição entra do mesmo jeito (coluna Imagem vazia ou foto sem acesso) e o erro vai para o log. Se a planilha falhar depois do upload, o arquivo vai para a lixeira.
- Coluna nova **Imagem** (Q) com o id do arquivo. Planilha antiga ganha os títulos que faltam na primeira gravação.

No site:

- `?action=teams` devolve `imageId` só para equipe `confirmado`. O `driveImageUrl.ts` monta `https://lh3.googleusercontent.com/d/<id>=w600` só quando o id tem o formato do Drive (`[A-Za-z0-9_-]`, 20 ou mais caracteres); a entidade `Team` só aceita URL `https` ou `data:image`. A CSP libera esse host só no `img-src`.
- Os cards (`TeamCardFrame`) mostram a imagem inteira (`object-fit: contain`) sobre uma cópia desfocada dela, com `loading="lazy"`, `decoding="async"`, largura e altura fixas e alt "Foto ou logo da equipe X". Sem foto, ou se ela não carregar, o card mostra as iniciais do

nome.

- **Moderação é a confirmação:** nada aparece antes de alguém da organização olhar a foto e marcar `confirmado`. Tirar do ar = apagar a célula **Imagem** (o card volta para as iniciais na hora). Detalhes em `apps-script/README.md`, seções 5 e 7.
- Em `npm run dev` sem endpoint, o `InMemoryRegistrationGateway` grava as equipes aceitas no `SimulatedTeamStore` (memória + `sessionStorage`), com a foto como data URL, e o `SimulatedTeamRepository` alimenta `/equipes` e a LP.

10. Identidade e assets

10.1 Logos e imagens

- **Logo HackaRun:** `src/assets/brand/hackarun-logo.png`, a imagem oficial. Nunca redesenhe; os raios em volta são desenhados por código (`motion/animators/hero/lightning/`) num canvas 2D, só com o hero visível.
- **Parceiros:** `unicatolica-white.png` e `capim-dourado-white.png` (versões brancas oficiais) aparecem discretas no hero e grandes na faixa "Quem faz acontecer" antes do rodapé (`components/partners/`).
- **Compartilhamento:** `public/og-image.jpg` (1200×630), usada no Open Graph e no Twitter Card.
- Imagens em `src/assets/` são otimizadas pelo Astro no build. Pedidos de arte em `docs/design/asset-brief.md`.
- **Fotos das equipes** não ficam no repositório: vêm do Drive da organização pelo CDN do Google (seção 9.6).
- **Som:** o site não tem nenhum. O botão de som, os sons sintetizados e o script que os gerava saíram a pedido da organização em 12/10/2026 (changelog, `[Unreleased]`).

10.2 Paleta e tipografia

A paleta oficial está em `src/presentation/styles/tokens.css` (detalhes na design spec). Fontes self-hosted, só o subset latin, declaradas no `BaseLayout.astro`:

- **Orbitron** (títulos, botões, números) em `src/assets/fonts/orbitron-pt-latin-wght.woff2`. A original tem um til que, em título grande, parece acento grave ("NÀO"). O script `python scripts/fonts/patch_orbitron_tilde.py` redesenha o til em degraus (Ã Õ ã õ Ñ ñ) a partir da fonte do Fontsource. Requer `fonttools` e `brotli` (`pip install fonttools brotli`) e o `node_modules` instalado.
- **Montserrat** (texto) e **JetBrains Mono** (HUD) vêm direto dos pacotes `@fontsource-variable/*`.

10.3 Assets dos PDFs

O pdfkit não lê woff2 variável. `python scripts/docs/build_pdf_assets.py` (requer `fonttools`, `brotli` e `Pillow`) gera instâncias estáticas das fontes da marca e versões menores dos logos em `scripts/docs/assets/`. Sem esses arquivos, os PDFs caem nas fontes padrão (Helvetica/Courier).

11. Convenções de código

- Código, nomes de arquivos e commits em **inglês**. Textos da interface e comentários em **português**.
- Comentário curto e objetivo, só quando o código não se explica sozinho. Nada de comentar o óbvio.
- POO: classes pequenas com uma responsabilidade. **Alvo de até 150 linhas por arquivo, teto de 250**: passou, quebre. O ESLint conta só linhas de código (sem branco nem comentário): **avisa acima de 200 e dá erro acima de 250**, inclusive nos `.astro` (o `<style>` conta). Componente grande vira subcomponentes na pasta da seção (ex.: `sections/npc/NpcTerminal.astro` com `TerminalBar` e `TerminalScreen`).
- Camadas: respeite a tabela da seção 6.1. A apresentação recebe **portas** da aplicação, nunca importa `@infrastructure/*` (o ESLint bloqueia); quem escolhe o adapter é o `src/composition/client.ts`.
- Classes e componentes em `PascalCase`, funções e variáveis em `camelCase`, constantes globais em `UPPER_SNAKE_CASE`.
- Imports por alias de camada: `@domain/*`, `@application/*`, `@infrastructure/*`, `@presentation/*`, `@shared/*`, `@composition/*`.
- `import type` para tipos (obrigatório pelo `verbatimModuleSyntax`). TypeScript no modo mais estrito (`noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`), sem `any`.
- CSS: use **só** os tokens de `styles/tokens.css` (cor da paleta é `var(--c-*)`, texto miúdo claro é `--ink-*`, campos usam `--control-*`); estilos com escopo dentro do `.astro`; media queries `min-width` (mobile first: o estilo base é o do celular e as larguras maiores somam por cima).
- Foco: nunca `outline: none`. Para esconder o contorno de um foco programático use `outline: 2px solid transparent`, que continua visível no modo de alto contraste (cores forçadas).
- Loop de animação infinito em CSS entra na lista do `styles/motion-pause.css` (seção 6.7).
- GLSL: um arquivo por objeto; constante espelhada com o `.ts` tem comentário dos dois lados.
- Acessibilidade e performance não são opcionais (checklist na seção 17).

12. Performance

12.1 Orçamento

Métrica	Meta
Lighthouse mobile	Performance >= 90 · Acessibilidade, Boas práticas e SEO 100
LCP (4G, celular médio)	< 2,5 s
CLS	< 0,05
INP	< 200 ms
JS inicial (gzip)	< 70 KB (three.js fica num chunk lazy)

Métrica	Meta
Fontes	3 woff2 latin (~90 KB no total), 2 com preload
Foto de equipe	Envio: meta 300 KB, teto 400 KB; no site, 600 px e lazy
Partículas	low 9k · medium 22k · high 42k

Situação em 11/10/2026 (Lighthouse mobile, build de produção, Brave headless, mediana de 3 execuções; para repetir, `node lighthouse.mjs <url> --runs=3` em `scripts/qa/`, seção 13.2):

Métrica	Antes da otimização	11/10/2026
Performance	66	95
Total Blocking Time (TBT)	1.600 ms	18 ms
Largest Contentful Paint (LCP)	2,9 s	2,6 s
First Contentful Paint (FCP)	não registrado	2,0 s
Speed Index	não registrado	2,5 s
Cumulative Layout Shift (CLS)	não registrado	0,048
Acessibilidade, Boas práticas e SEO	100	100

O TBT caiu com o boot fatiado e o 3D fora da thread principal (seção 12.2). O Lighthouse simula 4G lento e CPU 4x mais lenta, mais duro que o "celular médio" da meta: o LCP de 2,6 s ainda passa um pouco dos 2,5 s e continua como pendência (seção 19.2).

12.2 Técnicas aplicadas

- **HTML pronto no build:** o H1 do hero é o logo oficial e é o LCP: imagem otimizada pelo Astro, carregada sem lazy e com `fetchpriority` alta; CSS pequeno entra inline.
- **Boot fatiado depois do paint:** `App.boot()` espera o primeiro frame e roda a `BootQueue`, um passo curto por tarefa (seção 6.3): a primeira pintura não espera o JavaScript e o input não fica preso atrás do boot.
- **Setup sob demanda:** animações de entrada montam quando a seção se aproxima da tela (`ApproachWatcher`); os títulos usam o `LineSplit` próprio em vez do SplitText do GSAP, que cria um `Intl.Segmenter` caro ao carregar.
- **3D tardio e fora da thread principal:** o chunk do three.js só é importado depois do `load` e com o navegador ocioso. As formas são geradas num Web Worker enquanto os shaders compilam; o `StageCompiler` prepara um programa por tarefa com `KHR_parallel_shader_compile` (`compileAsync`), já no alvo de render certo, antes do primeiro frame.
- **Shader enxuto:** `defines` por cena removem ramos de formas ausentes; índices da paleta são constantes.
- **Render adiado do layout:** `content-visibility: auto` nas seções longe do topo, em `styles/deferred-sections.css` (importado pelo `base.css`, na layer `base`), com `contain-intrinsic-size` (altura do conteúdo, sem o padding da seção) medido em 390 e 1440 px. Mudou bastante o conteúdo de uma dessas seções? Meça de novo e atualize o valor. O `LayoutRefresher` recalcula os gatilhos quando a seção monta, e o scroll por âncora confere

o destino de novo ao chegar (`scripts/qa/anchorcheck.mjs` testa isso).

- **Aurora barata:** manchas em `radial-gradient` sem `filter: blur()`; cores interpoladas por `@property`.
- **Qualidade adaptativa** (seção 7.6), DPR limitado por tier, pausa com a aba oculta, raios do logo só com o hero visível e versão leve em telas de toque.
- **Fallbacks:** sem WebGL2 ou com `save-data`, o 3D não inicia e fica o brilho em CSS.
- **Página de equipes leve:** `/equipes` não baixa three.js, GSAP nem Lenis (`LightApp`, seção 6.3). As fotos vêm em 600 px do CDN do Google, com `loading="lazy"` e largura e altura fixas (sem pulo de layout), e a lista ao vivo só é buscada quando chega perto da tela.
- **Cache:** `/_astro/*` imutável (nome com hash) e HTML sempre revalidado (`public/_headers`).

13. QA

Os scripts ficam em `scripts/qa/`, com `package.json` próprio: Playwright e Lighthouse não entram no `npm ci` do projeto, só quem faz QA instala. Guia completo de cada script em `scripts/qa/README.md`.

```
cd scripts/qa
npm install
npx playwright install chromium   # só se a máquina não tiver Brave nem Chrome
```

O navegador sai de `BRAVE_PATH`, depois `CHROME_PATH`, depois do caminho padrão do Brave ou do Chrome no sistema e, por último, do Chromium do Playwright. O navegador instalado roda com **GPU real** (`--enable-gpu`, `--ignore-gpu-blocklist` e `--use-angle=d3d11` no Windows): sem isso o headless renderiza por software e o 3D sempre cai no tier `low`. Rode contra o build de produção (`npm run build && npm run preview`), passando a URL como primeiro argumento; as saídas vão para `scripts/qa/out/` (fora do Git).

13.1 Visual

Script	Uso
shots.mjs	node shots.mjs <url> 390 844 out/m390 "#sobre,#missoes@0.5" --mobile
anchorcheck.mjs	node anchorcheck.mjs <url>: links do header e CTAs param no alvo
overflow.mjs	node overflow.mjs <url> --width=360 --width=390: nada passa da tela

- Capturas por seção em 360/390, 768, 1024, 1366/1440 e 1920 px, com emulação de celular (`--mobile`: `isMobile`, toque, DPR 2) nas larguras pequenas.
- O `shots.mjs` espera a classe `is-experience-ready` no `<html>` (o 3D desenhou) e lista os erros do console no fim.
- Rode também com `?quality=low` e `?quality=high` na URL, com "reduzir movimento" ligado e com "Pausar animações" ligado.
- Mexeu em equipes, header ou rodapé: confira também `/equipes`, vazia e com equipes (em dev, inscreva equipes no modo simulado com fotos variadas: logo escura transparente, logo larga, foto grande, retrato).

- Âncoras: o `anchorcheck.mjs` clica nos links e confere se o destino parou na folga do header (`scroll-padding-top`); as seções com render adiado não podem errar o alvo. `anchorcheck` e `overflow` saem com código 1 quando falham.

13.2 Performance

Script	Uso
vitals.mjs	node vitals.mjs <url> --mobile: LCP, CLS, load, tempo até o 3D, em 4G e sem limite
longtasks.mjs	node longtasks.mjs <url> --runs=3: tarefas longas com CPU 4x e 4G lento, com mediana
trace.mjs	node trace.mjs <url>: trace até o 1º paint, downloads e passos boot:*
lighthouse.mjs	node lighthouse.mjs <url> --runs=3: Lighthouse mobile (--desktop) com mediana
lighthouse-summary.mjs	node lighthouse-summary.mjs out/lighthouse/*.json: resume relatórios já gravados

- Conte e atribua as tarefas longas (trace do Chrome) em vez de olhar só o tempo absoluto.
- As medições oscilam: rode várias vezes e compare medianas (os scripts de performance já imprimem a mediana).
- Script novo usa `lib/browser.mjs` e `lib/cli.mjs` (navegador, perfis de rede, argumentos e pasta de saída) e nunca caminho fixo da máquina de alguém.

14. Commits (Conventional Commits)

Formato: `<tipo>(<escopo>): <descrição curta no imperativo, em inglês>`

Tipo	Quando usar
feat	Nova funcionalidade
fix	Correção de bug
perf	Melhoria de performance
refactor	Mudança de código sem alterar comportamento
style	Formatação (sem mudar lógica)
test	Testes
docs	Documentação
build	Build e dependências
ci	Pipeline de CI
chore	Tarefas gerais
revert	Reverter commit

Escopos: `domain`, `application`, `infra`, `ui`, `sections`, `experience`, `motion`, `a11y`, `perf`, `seo`, `content`, `docs`, `ci`, `deps`, `config`, `tests`.

```
feat(sections): add prizes section with locked rewards
fix(experience): stop render loop when tab is hidden
content(faq): add question about team size        <- inválido: "content" é escopo, não tipo
docs(content): update schedule for day 14          <- correto
feat(domain)!: require team size rule              <- "!" = breaking change
```

O **commitlint** valida a mensagem no `commit-msg` e o **lint-staged** roda ESLint e Prettier no `pre-commit` (Husky). Mudanças em `package.json`, `docs/stack-manifest.json` ou neste guia regeneram os PDFs automaticamente.

15. Branches e Pull Requests

1. Atualize a `main` e crie a branch: `feat/<assunto>`, `fix/<assunto>`, `docs/<assunto>`.
2. Commits pequenos, no padrão acima.
3. `npm run validate` antes de abrir o PR.
4. Abra o PR pra `main` usando o template (checklist). A CI roda lint, formatação, testes e build; a Cloudflare gera uma URL de preview.
5. Pelo menos 1 revisão. Merge com **squash**, com título no padrão Conventional Commits.

Recomendado: proteger a `main` no GitHub (Settings -> Branches), exigindo PR e CI verde.

16. Deploy na Cloudflare Pages (grátis)

1. Crie uma conta gratuita em `dash.cloudflare.com`.
2. **Workers & Pages -> Create -> Pages -> Connect to Git** e escolha `Marcos-Codfy/lp-hackarun`.
3. Framework preset **Astro**. Build command `npm run build`. Output directory `dist`.
4. Variáveis de ambiente: `NODE_VERSION=24`, `PUBLIC_SITE_URL=https://<seu-projeto>.pages.dev`, `PUBLIC_REGISTRATION_ENDPOINT=<url do Web App>`.
5. **Save and Deploy**. Cada push na `main` publica em produção; cada PR ganha uma URL de preview.
6. (Opcional) Domínio próprio em **Custom domains**.

Cache e headers de segurança ficam em `public/_headers`.

Opção	Por que não foi a escolhida
Vercel (Hobby)	Termos do plano gratuito proíbem uso comercial/patrocinado; banda limitada a 100 GB/mês
GitHub Pages	Banda sugerida de 100 GB/mês, sem headers customizados e sem preview por PR
Cloudflare Pages	Escolhida: banda e requisições ilimitadas pra estático, CDN com servidores no Brasil, preview por PR

17. Checklist antes do merge

- [ ] `npm run validate` passou (e `npm run format:check`).
- [ ] Testei no celular (360 e 390 px) e no desktop (1440 px), com capturas das seções alteradas (`scripts/qa/shots.mjs`), sem scroll horizontal (`overflow.mjs`) e com as âncoras no alvo (`anchorcheck.mjs`).
- [ ] Mexeu no 3D: conferi com `?quality=low` e `?quality=high`, sem erro no console.
- [ ] Navegação só por teclado funciona; foco visível.
- [ ] Ativei "reduzir movimento" no sistema e a página continua utilizável.
- [ ] "Pausar animações" para tudo o que se mexe sozinho (loop novo em CSS entrou no `motion-pause.css`).
- [ ] Sem texto com contraste baixo; imagens com `alt`.
- [ ] Nenhum arquivo passou de 250 linhas (o ESLint avisa acima de 200); nenhum `console.log` esquecido.
- [ ] Lighthouse mobile sem regressão (performance >= 90; acessibilidade, boas práticas e SEO em 100).
- [ ] Mexeu em `apps-script/`: a nova versão do Web App sobe junto com o deploy do site (seção 9).
- [ ] Mudança anotada em **[Unreleased]** no `CHANGELOG.md` (seção 18).

18. Versionamento

SemVer em `package.json` (`MAJOR.MINOR.PATCH`). Tags `vX.Y.Z` nos marcos (ex.: `v1.0.0` no lançamento das inscrições). "v1" e "v2" no projeto são gerações do design, não versões: até as inscrições abrirem, tudo é `0.x`.

O histórico fica no `CHANGELOG.md`, no formato Keep a Changelog 1.1, em português:

- Toda mudança que entra na `main` ganha uma linha em **[Unreleased]**, no grupo certo: `Added` , `Changed`, `Removed`, `Fixed`, `Performance` ou `Security`.
- Ao fechar uma versão: troque **[Unreleased]** pelo número e pela data (`AAAA-MM-DD`), suba o `version` do `package.json` (`npm version X.Y.Z --no-git-tag-version`), faça o commit e crie a tag `vX.Y.Z` nele. Os links de comparação no fim do changelog dependem das tags.
- Antes da 1.0.0, funcionalidade nova ou mudança que quebra sobe o MINOR; correção e ajuste sobem o PATCH.

18.1 Histórico

Data	Versão	Marco
10/10/2026	0.1.0	Fundação: Astro + TypeScript estrito, Clean Architecture/DDD, tokens e layout base
10/10/2026	0.2.0	Site v1: seções, 3D de partículas, inscrição via Apps Script, testes, docs e CI
11/10/2026	0.3.0	v2 cinematográfica: identidade oficial, pós-processamento, LGPD, performance, QA e a11y

11/10/2026, v2 aprovada e publicada:

- Design v2 cinematográfico **aprovado**: mudanças visuais grandes daqui pra frente passam por nova aprovação.
- Código publicado no GitHub: https://github.com/Marcos-Codfy/lp-hackarun (branch `main`).
- Um deploy temporário serviu pra validar o design com a organização. A hospedagem oficial continua a Cloudflare Pages (ADR 0003), ainda por conectar.
- Lighthouse mobile: Performance 66 -> **95**, TBT 1.600 -> 18 ms, LCP 2,9 -> 2,6 s; Acessibilidade, Boas práticas e SEO em 100 (detalhes na seção 12.1).
- O README virou a página de apresentação do projeto (portfólio): `README.md` em português e `README.en.md` em inglês.
- Changelog criado (`CHANGELOG.md`), com as versões 0.1.0 a 0.3.0 derivadas do histórico do Git.

12/10/2026, ajustes pedidos pela organização (em [Unreleased]):

- **Sem som:** saíram o botão de som, o cue do globo, o som de digitação, as portas `AudioCue` e `TickSound`, os eventos `sound:changed` e `ui:type`, `public/audio/` e `scripts/audio/render_cues.py`.
- **Clímax mais calmo:** carga de 0,9 s (era 1,5 s), pulso suave no lugar do estouro, sem flash branco nem tremor de câmera, anel menor e brilho com teto. A barra do CTA enche junto com a carga (seção 7.5).
- **Página `/equipes`:** cada equipe confirmada com a foto ou a logo que sobe na inscrição, guardada no Drive da organização (seção 9.6, ADR 0007). A seção de equipes da LP virou uma chamada para a página, e "Equipes" entrou no header, no menu e no rodapé.
- **Consentimento `2026-10-v2`:** inclui a autorização de imagem; o Apps Script só aceita a versão vigente.
- **Textos:** "Final no Capim Dourado Shopping" (no lugar de "Final no telão"), "UNICATÓLICA" na faixa (no lugar de "Palmas–TO"), "Da sala de aula ao palco." no título de "O evento" e chamada nova dos prêmios.
- **Saíram:** o "Adicionar à agenda" da programação (o `.ics` segue na área de inscrição), o aviso "Corra contra o relógio, não pelos corredores" dos NPCs (a regra vai ser dita no dia) e o link do repositório no rodapé.

19. Pendências

19.1 Da organização

- [ ] URL do Web App de inscrições (`PUBLIC_REGISTRATION_ENDPOINT`).
- [ ] Prazo das inscrições (`registration.closesAt`) e capacidade de equipes (`registration.capacity`), repetidos nas Propriedades `CLOSES_AT` e `CAPACITY` do Apps Script (seção 9.1).
- [ ] Tamanho mínimo e máximo da equipe (`registration.teamSize`, hoje 2 a 5 provisório; igual ao `TEAM_SIZE` do `apps-script/Validation.gs`).
- [ ] Inscrição individual quando as vagas de equipe lotarem: hoje fecha pra todo mundo (seção 9.1); confirmar se é isso mesmo.
- [ ] Prêmios (`prizes.json`, os 3 estão "a revelar") e link do regulamento oficial.

- [ ] Nomes de instrutores, mentores, palestrante e banca (`speakers.json`, todos "a revelar").
- [ ] E-mail do encarregado de dados (LGPD) para o aviso em `/privacidade` (hoje aparece como "pendente").
- [ ] Revisão jurídica do texto do aviso de privacidade. Mudou o texto? Suba o `CONSENT_VERSION` (seção 9.3).
- [ ] Conta Google institucional dona da planilha e do Apps Script (cotas maiores, e os dados ficam com a instituição, não com uma pessoa).
- [ ] Site oficial do Capim Dourado para o link da faixa de parceiros (`partners.json`, hoje `null`).
- [ ] Local da caça aos NPCs (universidade ou shopping).
- [ ] Publicar a nova versão do Apps Script (os quatro `.gs`), rodar `prepararFotos` uma vez para autorizar o Drive e subir junto com o deploy do site (seção 9).
- [ ] Confirmar que a conta institucional permite link público ("Qualquer pessoa com o link · Leitor") nos arquivos da pasta das fotos; senão as equipes aparecem só com as iniciais (seção 9.6).
- [ ] FAQ "Vou precisar correr pelos corredores?" (`faq.json`): repete a regra de caminhar entre as estações, que a organização vai passar só no dia. Confirmar se a pergunta sai também.
- [ ] Logos em SVG (opcional; as versões PNG oficiais já estão no site).
- [ ] Domínio próprio (opcional).

19.2 Técnicas

- [x] Lighthouse mobile >= 90 em performance: **95** em 11/10/2026 (seção 12.1).
- [x] Versionar os scripts de QA (capturas, vitals, tarefas longas, trace, âncoras, overflow e Lighthouse) em `scripts/qa/`.
- [x] Remover dependências da v1 que não são mais usadas (`@fontsource-variable/big-shoulders` e `inter`).
- [x] Publicar o código no GitHub (`main`).
- [x] README de apresentação (pt-BR e inglês) e `CHANGELOG.md`.
- [ ] Conectar a Cloudflare Pages (seção 16) e definir `PUBLIC_SITE_URL` com a URL final.
- [ ] Excluir o deploy temporário de validação depois da aprovação final.
- [x] Subir o `version` do `package.json` para `0.3.0` e criar as tags `v0.1.0` (`160bcda`), `v0.2.0` (`c5cf8c1`) e `v0.3.0` (commit que publica o changelog). Os links do `CHANGELOG.md` dependem delas.
- [ ] LCP do Lighthouse mobile abaixo de 2,5 s (hoje 2,6 s; meta da seção 12.1).
- [ ] Medir de novo o Lighthouse da LP e medir o de `/equipes` no build de produção. Em dev, `/equipes` deu Acessibilidade, Boas práticas e SEO 100; a Performance ainda não foi medida.
- [ ] Testar o caminho das fotos no Apps Script de verdade (autorização do Drive, compartilhamento num domínio institucional e o `lh3.googleusercontent.com` servindo o link público). Hoje só os dublês dos testes cobrem.
- [ ] Recapturar as imagens do README (`docs/media/`) e atualizar os textos alternativos: são de antes dos ajustes de 12/10 (botão de som no header, títulos antigos de "O evento" e da grande final, "Adicionar à agenda" na programação e a seção de equipes antiga).
- [ ] Levar `/privacidade` e `/404` para o `PageLayout` leve (seção 6.3), se fizer sentido.

- [ ] Proteger a `main` no GitHub, exigindo PR e CI verde (seção 15).
- [ ] Cloudflare Turnstile no formulário, quando a organização gerar as chaves (`apps-script/README.md`).

20. Documentação gerada

`npm run docs:pdf` gera em `docs/pdf/`:

- `HackaRun_LP_Stack_e_Padroes.pdf`: stack, bibliotecas (lidas do `package.json`), recursos, padrões e conceitos (só os nomes, vindos de `docs/stack-manifest.json`).
- `HackaRun_LP_Guia_do_Projeto.pdf`: este guia.

Os PDFs usam as fontes da marca (Orbitron, Montserrat e JetBrains Mono, em `scripts/docs/assets/`). Caracteres que essas fontes não têm (setas, sinais de comparação, desenho de caixa) viram equivalentes em ASCII. Se `DOCS_OUT_DIR` estiver definido no `.env`, os PDFs e este guia também são copiados pra essa pasta.